PRESENTASI TUGAS KELOMPOK

Analisis & Implementasi Perbandingan

Layered vs Non-Layered Web Application Architecture

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)

1. Diagram Pemisahan Folder

- Hubungan HTML, CSS, JS, & PHP
- pada Non-Layered vs 3-Layer

2. Screenshot Code & Web

- Bukti Kode Program & Tampilan
- Kedua Pendekatan Arsitektur

3. Tabel Pengujian & Unggulan

- Perbandingan Kecepatan, Tim,
- Penelusuran Error, & Alasan

Studi Kasus: Customer Management System

6 Fitur Wajib Aplikasi & Desain Tabel Database MySQL (customer_management.sql)

6 Fitur Wajib yang Diimplementasikan

1. Tambah Pelanggan

Memasukkan data pelanggan baru (nama, email, telepon, alamat).

2. Tampil Pelanggan

Menampilkan seluruh daftar data pelanggan perusahaan.

3. Edit Pelanggan

Mengubah informasi pelanggan yang sudah terdaftar.

4. Hapus Pelanggan

Menghapus data pelanggan dari database MySQL.

5. Pencarian Pelanggan

Mencari pelanggan berdasarkan Nama atau Email.

6. Validasi Data

Memeriksa kelengkapan kolom dan keunikan alamat email.

File Database SQL: customer_management.sql

```
CREATE DATABASE IF NOT EXISTS customer_management;
USE customer_management;

CREATE TABLE pelanggan (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    telepon VARCHAR(25) NOT NULL,
    alamat TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 5 Data Awal Pelanggan:
INSERT INTO pelanggan VALUES
('Andi Pratama', 'andi@nusantara.co.id', ...),
('Budi Santoso', 'budi@teknologimaju.id', ...),
('Citra Lestari', 'citra@kreasidigital.com', ...),
('Dewi Anggraini', 'dewi@sinergiglobal.co.id', ...),
('Eko Prasetyo', 'eko@mitraniaga.id', ...);
```

Diagram Arsitektur 1: Non-Layered Architecture

Struktur Folder & Hubungan HTML, CSS, Validasi PHP, dan SQL Menyatu dalam 1 File (index.php)

DIAGRAM ARSITEKTUR 1: NON-LAYERED (STRUKTUR FILE & HUBUNGAN KOMPONEN)

Tidak Ada Pemisahan File/Folder Layer: HTML, CSS, Logika PHP, dan Query SQL Menyatu di 1 File (index.php)

STRUKTUR FOLDER & FILE (NON-LAYERED)

```
[Folder] 1-non-layered-app/
 └── [File] index.php   (HANYA 1 FILE!)
      ├── [Baris 16-39]  Koneksi PDO MySQL
      ├── [Baris 44-160] Validasi PHP & Query SQL
      ├── [Baris 168-200] Kode Desain <style> CSS
      └── [Baris 202-304] Kode Tampilan HTML
```

Ciri Struktur File Non-Layered:

1. Satu File untuk Semua

HTML, CSS, PHP, dan SQL tidak dipisah ke dalam folder layer.

2. Ketergantungan Tinggi

Mengubah desain HTML/CSS harus mengedit file yang berisi SQL.

3. Sulit Dikerjakan Tim

Jika beberapa orang mengedit index.php bersamaan, rawan bentrok.

4. Tanpa REST API

Form HTML langsung disubmit ke index.php (Full Page Reload).

ISI DI DALAM FILE: 1-non-layered-app/index.php (MENYATU)

BAGIAN TAMPILAN: HTML + CSS (Menyatu di Bagian Bawah index.php)

- Tag <style> CSS mengatur warna & tabel langsung di dalam index.php
- Tag <form> HTML & tabel <table> bercampur dengan <?php foreach (...) ?>

BAGIAN PROSES BISNIS: Validasi PHP (Menyatu di Bagian Tengah index.php)

- Blok if ($_SERVER['REQUEST_METHOD'] === 'POST') menerima input form
- Memeriksa kelengkapan kolom kosong & format email langsung di file yang sama

BAGIAN AKSES DATA: Koneksi PDO & Query SQL (Menyatu di index.php)

- `new PDO('mysql:host=localhost;dbname=customer_management', 'root', '')`
- Query SELECT, INSERT, UPDATE, DELETE ditulis langsung bersama kode form

Query SQL Langsung

Data Tabel pelanggan

DATABASE MYSQL: customer_management (File: database/customer_management.sql)

- Tabel `pelanggan`: id (PK), nama, email (UNIQUE), telepon, alamat, created_at
- **Diakses langsung dari file tampilan index.php tanpa pemisahan folder layer**

Screenshot Code Aplikasi 1 (Non-Layered)

Bukti Kode 1-non-layered-app/index.php: Koneksi DB, Validasi, Query SQL, CSS, & HTML Menyatu

SCREENSHOT CODE 1: KONSEP NON-LAYERED ARCHITECTURE (1 FILE MONOLITIK)

File: 1-non-layered-app/index.php — Koneksi Database, Validasi Proses Bisnis, Query SQL, CSS, dan HTML Menyatu dalam 1 File

1-non-layered-app/index.php (Seluruh Kode Menyatu dalam 1 File)

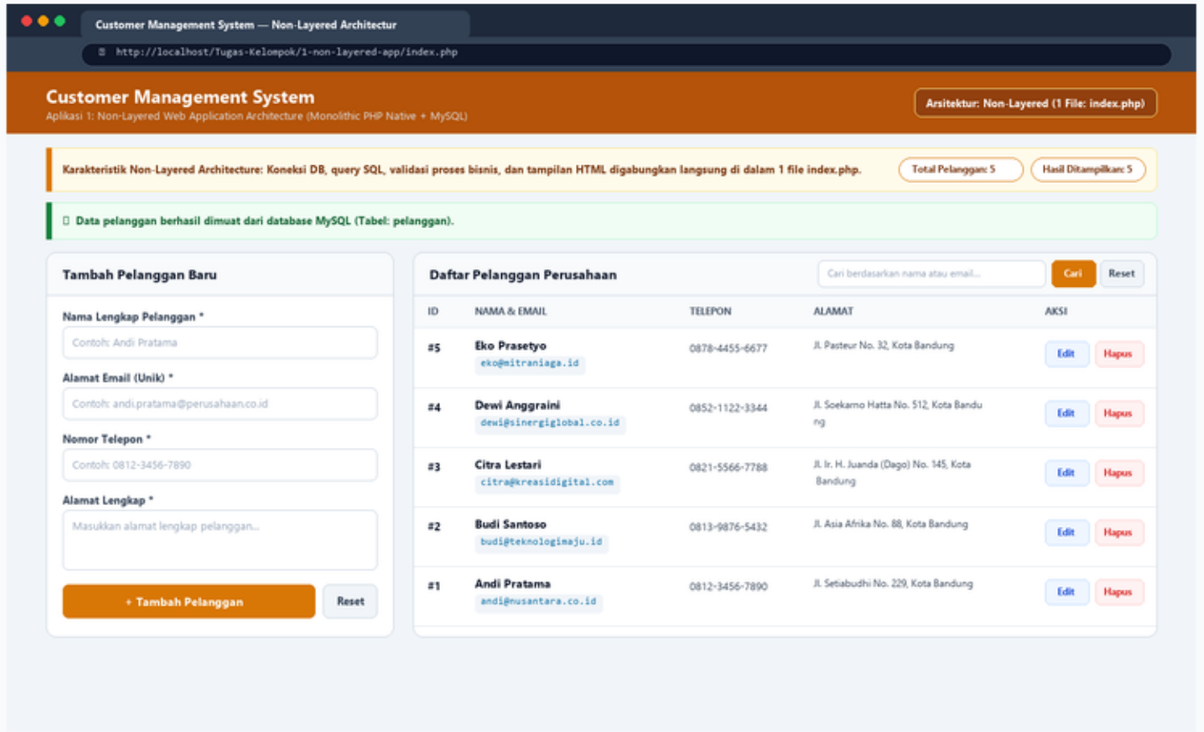
NON-LAYERED (MONOLITIK)

```
// ========================================================================
// BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL (Di dalam index.php)
// ========================================================================
$koneksi = new PDO("mysql:host=localhost;dbname=customer_management", "root", "");

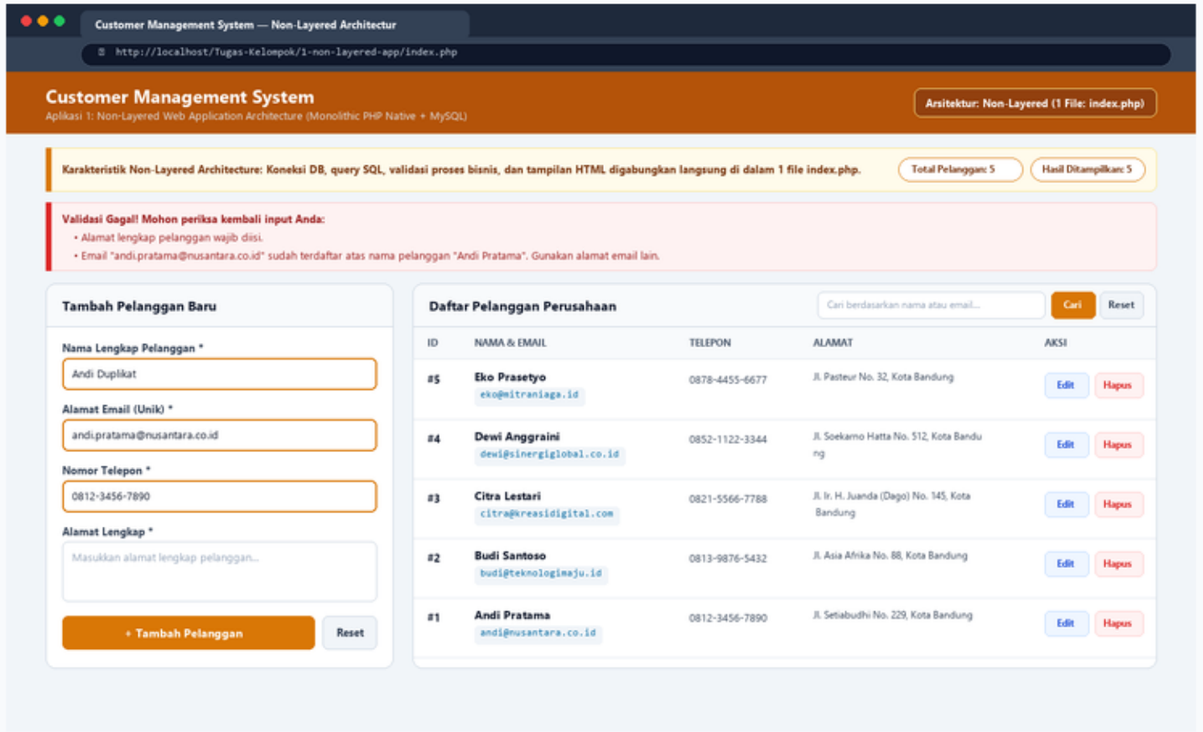
// ========================================================================
// BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD (Menyatu di index.php)
// ========================================================================
if (isset($_GET['hapus'])) {
    $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
    $stmt->execute([(int) $_GET['hapus']]);
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // A. Validasi Kelengkapan Data
    if ($nama === '' || $email === '' || $telepon === '' || $alamat === '') {
        $pesan_error[] = "Semua kolom pelanggan wajib diisi.";
    }
    // B. Validasi Keunikan Email (Langsung query SQL di file tampilan)
    $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
    $stmtCek->execute([$email]);
    if ($stmtCek->fetch()) {
        $pesan_error[] = "Email sudah terdaftar di database!";
    }
    // C. Eksekusi Query INSERT / UPDATE Langsung
    $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?,?,?,?)");
    $stmt->execute([$nama, $email, $telepon, $alamat]);
}
// Query Tampil & Pencarian Pelanggan
$daftar_pelanggan = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC")->fetchAll();
?>
<!-- ========================================================================
     BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung di bawah kode PHP)
     ======================================================================== -->
<!DOCTYPE html><html lang="id"><head><style> /* Kode CSS di sini */ </style></head>
```

Screenshot Tampilan Web Aplikasi 1 (Non-Layered)

Tampilan Daftar Pelanggan & Validasi Email Unik pada 1-non-layered-app/index.php



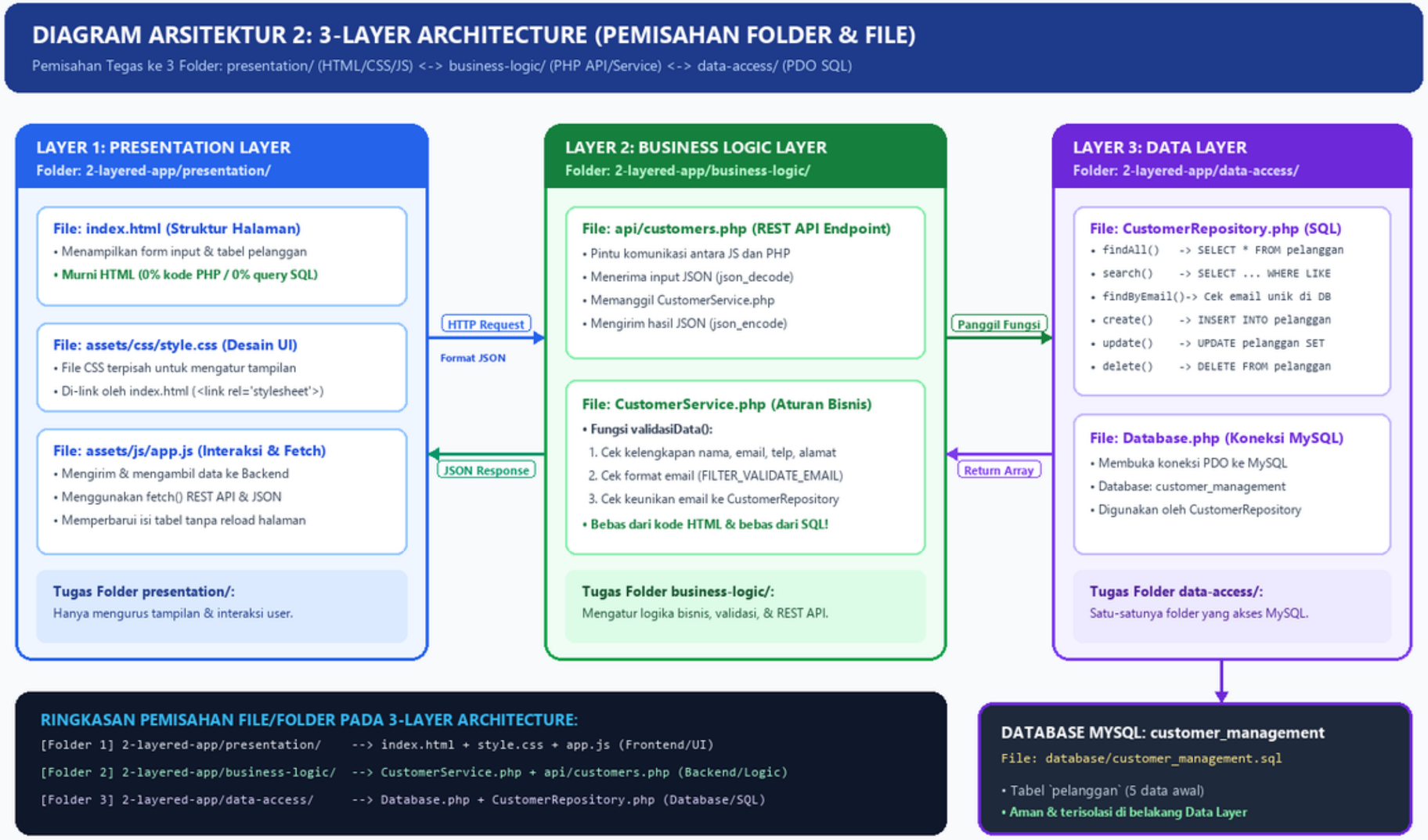
Tampilan Utama & Tambah Pelanggan (index.php)



Tampilan Validasi Kolom Kosong & Email Unik

Diagram Arsitektur 2: 3-Layer Architecture

Pemisahan File/Folder secara Tegas: presentation/ <-> business-logic/ <-> data-access/



Screenshot Code Aplikasi 2 (3-Layer Architecture)

Bukti Pemisahan Kode ke 3 Folder: app.js (Presentation), CustomerService.php (Logic), & CustomerRepository.php (Data)

SCREENSHOT CODE 2: KONSEP 3-LAYER ARCHITECTURE (TERPISAH 3 FOLDER LAYER)

Pemisahan Tegas: Presentation Layer (app.js) <-> Business Logic Layer (CustomerService.php & api) <-> Data Layer (CustomerRepository.php)

presentation/app.js — LAYER 1: PRESENTATION

```
1   // Folder: 2-layered-app/presentation/
2   // Murni Frontend (Tanpa PHP & Tanpa SQL)
8   const URL_API = '../business-logic/api/customers.php';
9
11  // 1. Ambil Data via GET REST API (JSON)
12  async function muatDataPelanggan(kataKunci = '') {
18    const response = await fetch(url);
19    const hasil = await response.json();
30    tbody.innerHTML = hasil.data.map(...);
45  }
46
48  // 2. Kirim Data via POST/PUT JSON
60  const response = await fetch(url, {
61    method: idEdit ? 'PUT' : 'POST',
62    headers: {'Content-Type':'application/json'},
63    body: JSON.stringify(dataKirim)
64  });
66  const hasil = await response.json();
```

CustomerService.php — LAYER 2: BUSINESS LOGIC

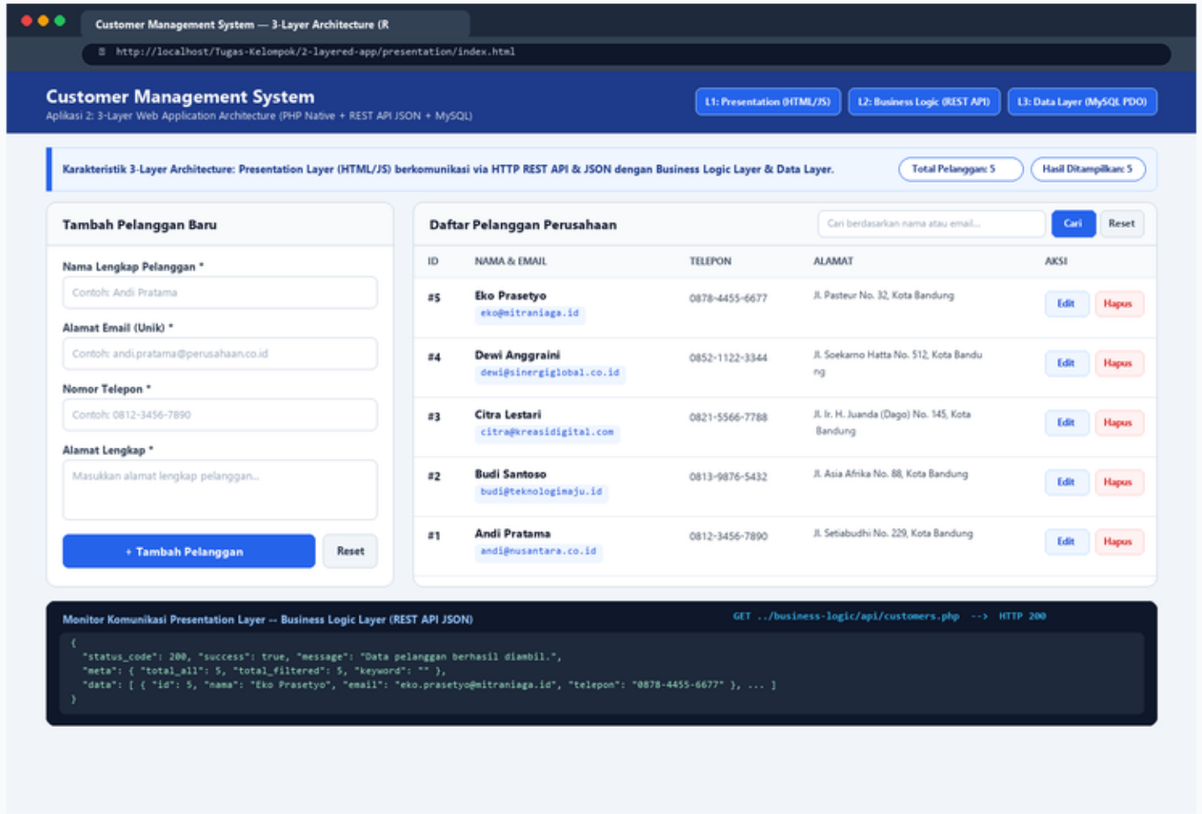
```
1   // Folder: 2-layered-app/business-logic/
2   // Aturan Bisnis, Validasi, & REST API JSON
14  class CustomerService {
56    public function createCustomer($input) {
58      $data = $this->rapikanInput($input);
59      $errors = $this->validasiData($data);
62      if (!empty($errors)) {
63        return ['status_code' => 422,
64                'success' => false,
65                'errors' => $errors];
67      }
70      $baru = $this->repo->create($data);
72      return ['status_code' => 201,
73              'success' => true,
74              'data' => $baru];
76    }
150   // Validasi kelengkapan & email unik
151   private function validasiData($data) {...}
180 }
```

CustomerRepository.php — LAYER 3: DATA LAYER

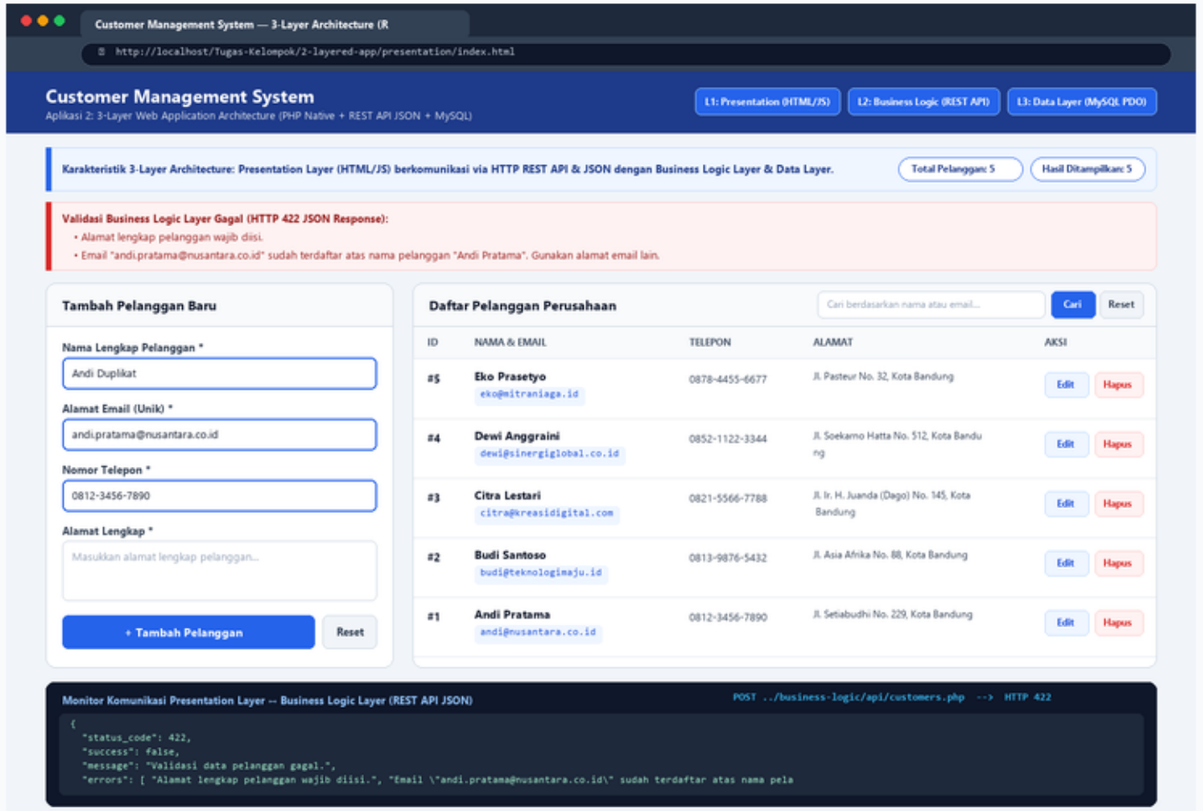
```
1   // Folder: 2-layered-app/data-access/
2   // Khusus Koneksi PDO & Query SQL MySQL
12  class CustomerRepository {
22    public function findAll() {
24      return $this->db->query(
25        'SELECT * FROM pelanggan ORDER BY id DESC'
26      )->fetchAll();
27    }
46    public function findByEmail($email) {
52      $stmt = $this->db->prepare(
53        'SELECT * FROM pelanggan WHERE email = ?'
54      );
57    }
60    public function create($data) {
62      $stmt = $this->db->prepare(
63        'INSERT INTO pelanggan VALUES (?,?,?,?)'
64      );
65      $stmt->execute([...]);
70    }
95  }
```

Screenshot Tampilan Web Aplikasi 2 (3-Layer REST API)

Tampilan Antarmuka Terpisah beserta Bukti Pertukaran Data REST API & JSON



Dashboard 3-Layer + Monitor JSON (HTTP 200 OK)



Respons Validasi Business Logic Layer (HTTP 422 JSON)

Perbandingan Alur Proses Request & Response

Ringkasan Cara Kerja di Balik Layar Setelah Demo Kedua Aplikasi

PERBANDINGAN ALUR EKSEKUSI: NON-LAYERED vs 3-LAYER ARCHITECTURE

Studi Kasus: Proses Tambah Pelanggan & Validasi Keunikan Email pada Customer Management System

PENDEKATAN 1: NON-LAYERED ARCHITECTURE (MONOLITIK)

Tahap 1 — Input Form HTML

Pengguna mengisi form di index.php lalu menekan tombol Submit.

Tahap 2 — HTTP POST Form-Urlencoded

Browser mengirim form ke index.php (memicu pemuatan ulang halaman).

Tahap 3 — Validasi Prosedural di index.php

Blok if ($_POST) memeriksa field kosong & menjalankan query cek email.

Tahap 4 — Eksekusi INSERT Langsung

File index.php langsung mengeksekusi query INSERT INTO pelanggan.

Tahap 5 — Query Ulang & Render HTML Utuh

File index.php menjalankan SELECT * lalu mengembalikan seluruh HTML.

PENDEKATAN 2: 3-LAYER ARCHITECTURE (REST API & JSON)

Tahap 1 — Presentation Layer (app.js)

Menangkap submit form & mengirim HTTP POST JSON via fetch() async.

Tahap 2 — REST API Controller (api/customers.php)

Menerima JSON payload & meneruskannya ke CustomerService.

Tahap 3 — Business Logic Layer (CustomerService.php)

Memvalidasi kelengkapan data, format email, & memanggil cek duplikasi.

Tahap 4 — Data Layer (CustomerRepository.php)

Menjalankan Prepared Statement findByEmail() & create() ke MySQL.

Tahap 5 — JSON Response (HTTP 201 / 422)

Backend mengirim JSON; Presentation Layer memperbarui tabel DOM.

Tabel Hasil Pengujian & Perbandingan Akhir

Perbandingan Unsur Uji Coba, Siapa yang Lebih Unggul, beserta Alasannya

Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Utama
1. Pemisahan File/Folder	Menyatu di 1 file index.php	Terpisah tegas di 3 folder layer	**3-Layer**	**Kode rapi & tanggung jawab tiap file jelas**
2. Kecepatan Buat Awal	Sangat cepat (cukup 1 file)	Lebih lama (buat API & 3 layer)	**Non-Layered**	**Praktis untuk aplikasi kecil/sederhana**
3. Individu vs Kerja Tim	Hanya cocok dikerjakan 1 orang	Bisa dibagi paralel ke 3 orang/tim	**3-Layer**	**Tiap anggota kerjakan folder berbeda tanpa bentrok**
4. Penelusuran Error (Bug)	Sulit dicari karena kode campur	Mudah dilacak sesuai foldernya	**3-Layer**	**Langsung tahu letak bug (UI / Logic / SQL)**
5. Kecepatan Respons UI	Lambat (Full Page Reload)	Cepat (Fetch JSON tanpa reload)	**3-Layer**	**Hanya bertukar data JSON yang ringan**
6. Keamanan Edit Desain	Rawan merusak kode PHP/SQL	Aman (HTML/CSS terpisah penuh)	**3-Layer**	**Ubah tampilan tidak menyentuh kode database**
7. Penambahan Fitur Baru	File index.php makin panjang	Tinggal tambah fungsi per layer	**3-Layer**	**Mudah dikembangkan tanpa merusak kode lama**
8. Reusabilitas (Mobile)	Tidak bisa (output berupa HTML)	Bisa dipakai Web & Android/iOS	**3-Layer**	**REST API menghasilkan format standar JSON**

Jawaban 5 Pertanyaan Diskusi & Kesimpulan

Ringkasan Analisis Akhir Perbandingan Layered vs Non-Layered Architecture

1. Mana yang lebih mudah dibuat?

Non-Layered lebih cepat dibuat di awal untuk aplikasi kecil/individu karena seluruh kode cukup ditulis dalam 1 file index.php.

2. Mana yang lebih mudah diperbaiki saat terjadi kesalahan (bug)?

3-Layer jauh lebih mudah ditelusuri karena bug langsung terisolasi sesuai foldernya (apakah di presentation/, business-logic/, atau data-access/).

3. Mana yang lebih mudah dikembangkan saat fitur bertambah?

3-Layer jauh lebih unggul karena mendukung kerja tim secara paralel dan endpoint REST API JSON-nya bisa dipakai ulang oleh aplikasi lain.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Kode tercampur aduk (spaghetti code), edit desain HTML rawan merusak query SQL, rawan bentrok saat kerja tim, dan sulit melacak bug.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk tugas individu atau web kecil sederhana, Non-Layered lebih cepat. Namun untuk sistem bisnis & kerja tim, 3-Layer mutlak lebih unggul.